SWEN90016

Software Processes and Management

Semester 1, 2026 Lecture 2

Faculty of Engineering and Information Technology
School of Computing and Information Systems

The University of Melbourne acknowledges the Traditional Owners of the unceded land on which we work, learn and live: the Wurundjeri Woi-wurrung and Bunurong peoples (Burnley, Fishermans Bend, Parkville, Southbank and Werribee campuses), the Yorta Yorta Nation (Dookie and Shepparton campuses), and the Dja Dja Wurrung people (Creswick campus).

The University also acknowledges and is grateful to the Traditional Owners, Elders and Knowledge Holders of all Indigenous nations and clans who have been instrumental in our reconciliation journey.

We recognise the unique place held by Aboriginal and Torres Strait Islander peoples as the original owners and custodians of the lands and waterways across the Australian continent, with histories of continuous connection dating back more than 60,000 years. We also acknowledge their enduring cultural practices of caring for Country.

We pay respect to Elders past, present and future, and acknowledge the importance of Indigenous knowledge in the Academy. As a community of researchers, teachers, professional staff and students we are privileged to work and learn every day with Indigenous colleagues and partners.

Please ask your questions about the SWEN90016 Lecture

Nobody has responded yet.

Hang tight! Responses are coming in.

Week 1 - Recap

- ✓ Understand Assignments and our expectations
- ✓ Understand key elements of a Project and why organisations use them
- ✓ Understand the foundational components of Project Management
- ✓ Understand key skills, responsibilities & activities of a Project Manager
- ✓ Understand key elements of how to manage Projects
- ✓ Exposure to some Project Management Methodologies

Week 1 - Recap

- ✓ Explore key drivers in why projects fail / succeed
- ✓ Understand how organisations select the best / right projects
- ✓ Understand the Project Initialisation process, Business Case structure and why organisations use them
- ✓ Explore various Investment techniques and financial models
- ✓ Understand responsibilities associated with building a Business Case and the accountable group / individual
- ✓ Understand what a Project Charter is and how it is used

Week 2

Intended Learning Objectives

- Formal Process
- A Prehistory of SDLCs
- Agile
- Agile Family
- Scrum
- PayPal Case Study

What is a Process??

Dictionary definition of process: *a series of actions or steps taken in order to achieve a particular end.*

A process is a series of progressive and interdependent steps by which an end is attained.

- Defined Process
- Empirical Process

A process model is a model (i.e. a simplified description) of the steps you need to take to achieve your goal.

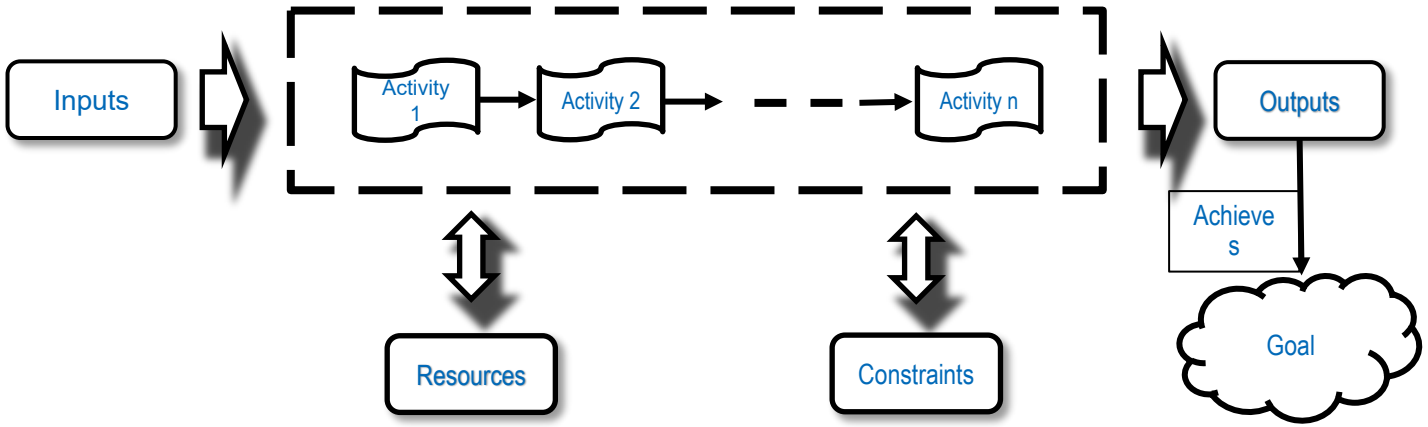
Scrum foundations – Video on Empirical v/s Defined Process [Readings Online in CANVAS]

Empirical vs defined process control [Readings Online in CANVAS]

Defined Process Control

A process with a well-defined set of steps. Given the same inputs, a defined process should produce the same output every time.

Great when in an environment with relatively low volatility that can be easily predicted; given the same inputs, a defined process should produce the same output every time based on its repeatability and predictability nature.

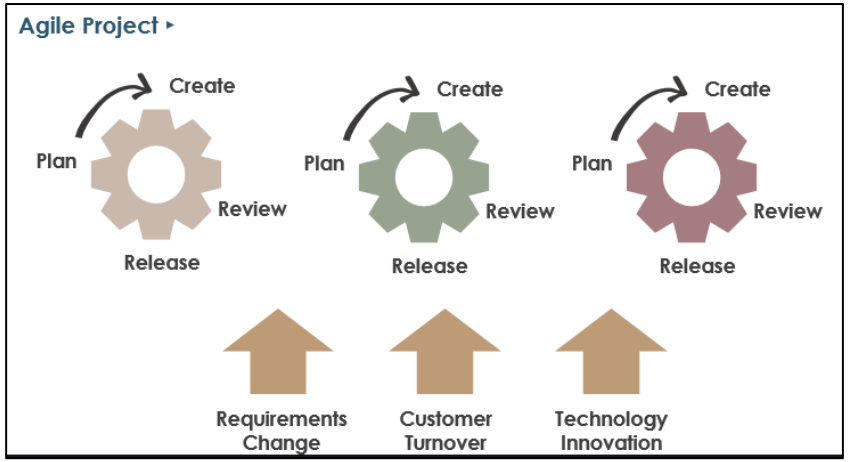


Empirical vs defined process control [Readings Online in CANVAS]

Empirical Process Control

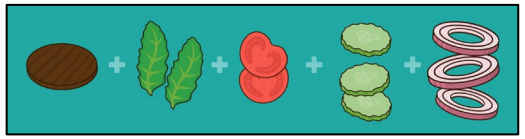
In empirical process control, you expect the unexpected. Empirical process control has the following characteristics:

- Learn as we progress
- Expect and embrace change
- Inspect and adapt using short development cycles
- Estimates are indicative only and may not be accurate

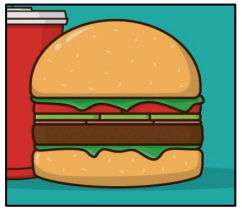


Empirical vs defined process control [Readings Online in CANVAS]

Defined vs Empirical



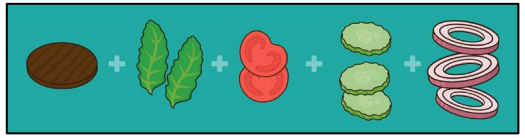
=

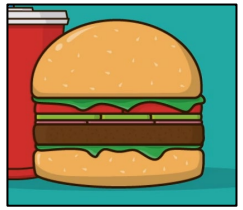


Defined / Repeatable Process

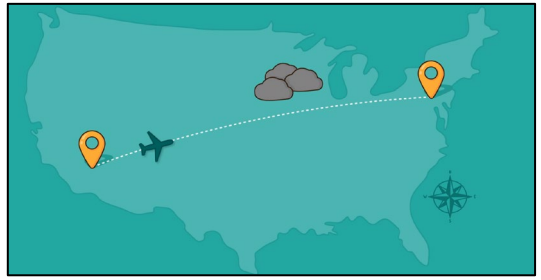
Scrum foundations [Readings Online in CANVAS]

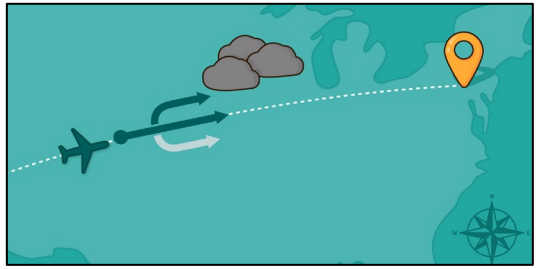
Defined vs Empirical

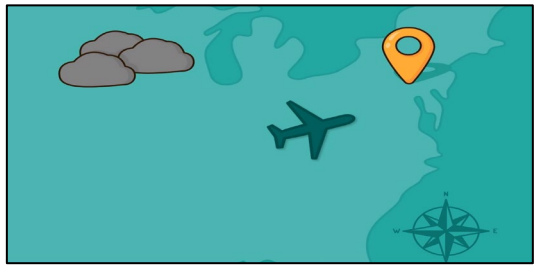




Defined / Repeatable Process







Scrum foundations [Readings Online in CANVAS]

What is a Process Model?

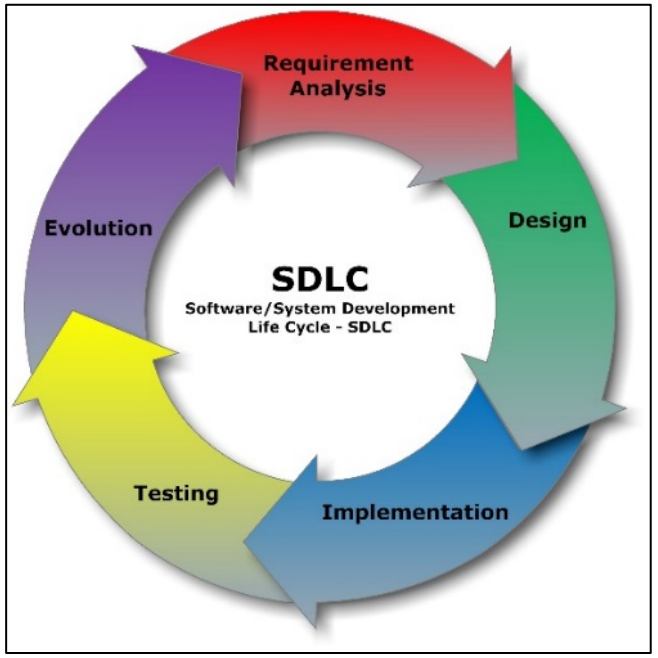
- A process model is a model (i.e. a simplified description) of the steps you need to take to achieve your goal.
- A software process model tells you which activities should happen when
  - helps you build what your customer wants and make sure it's right
- Good management enables all this to happen on time and on budget

Software Development Life Cycles (SDLCs) aka Processes

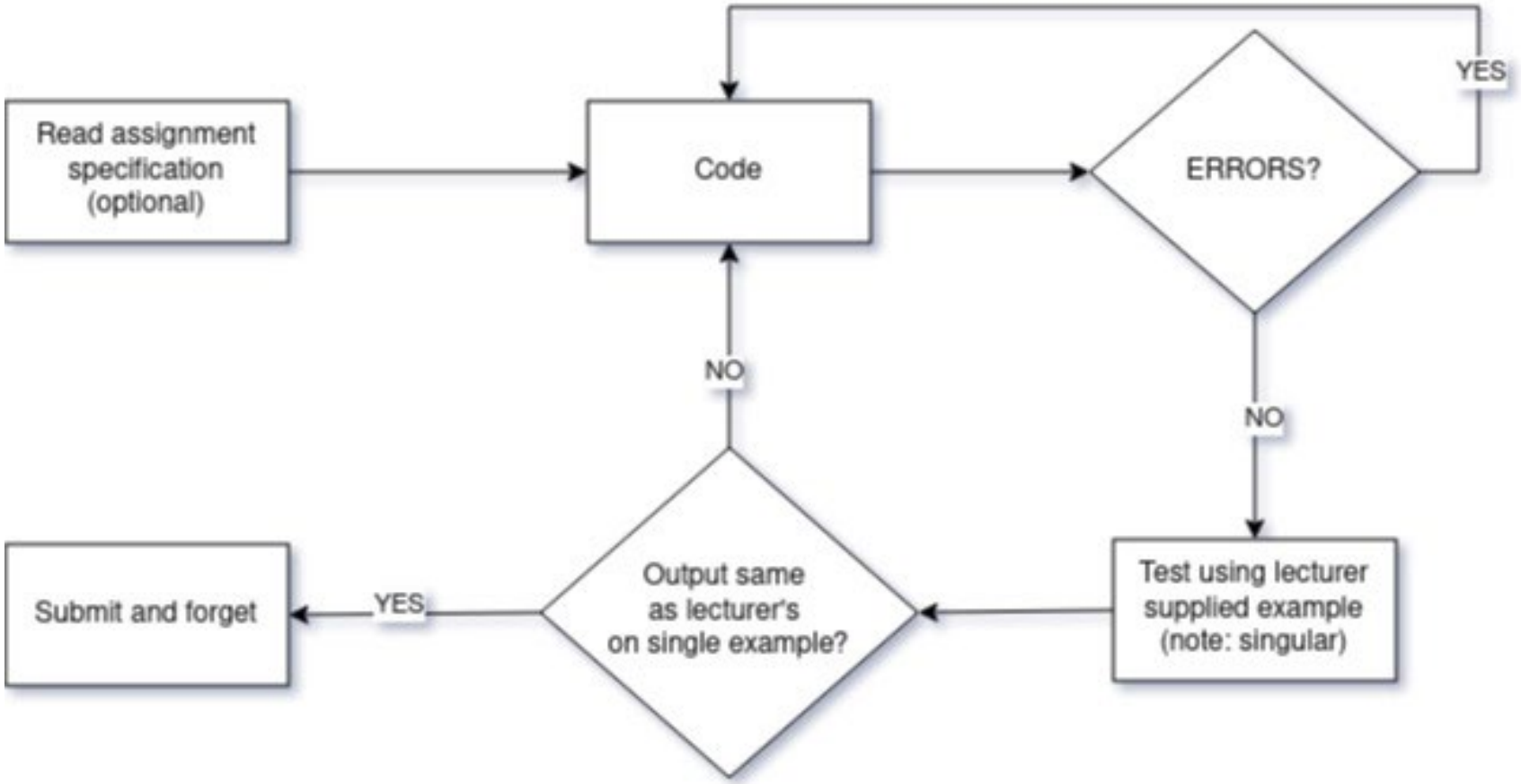
The systems development life cycle (SDLC), also referred to as the application development life-cycle, is a term used in systems engineering, information systems and software engineering to describe a ***process*** for planning, creating, testing and deploying an information system.

Activities in SDLC:

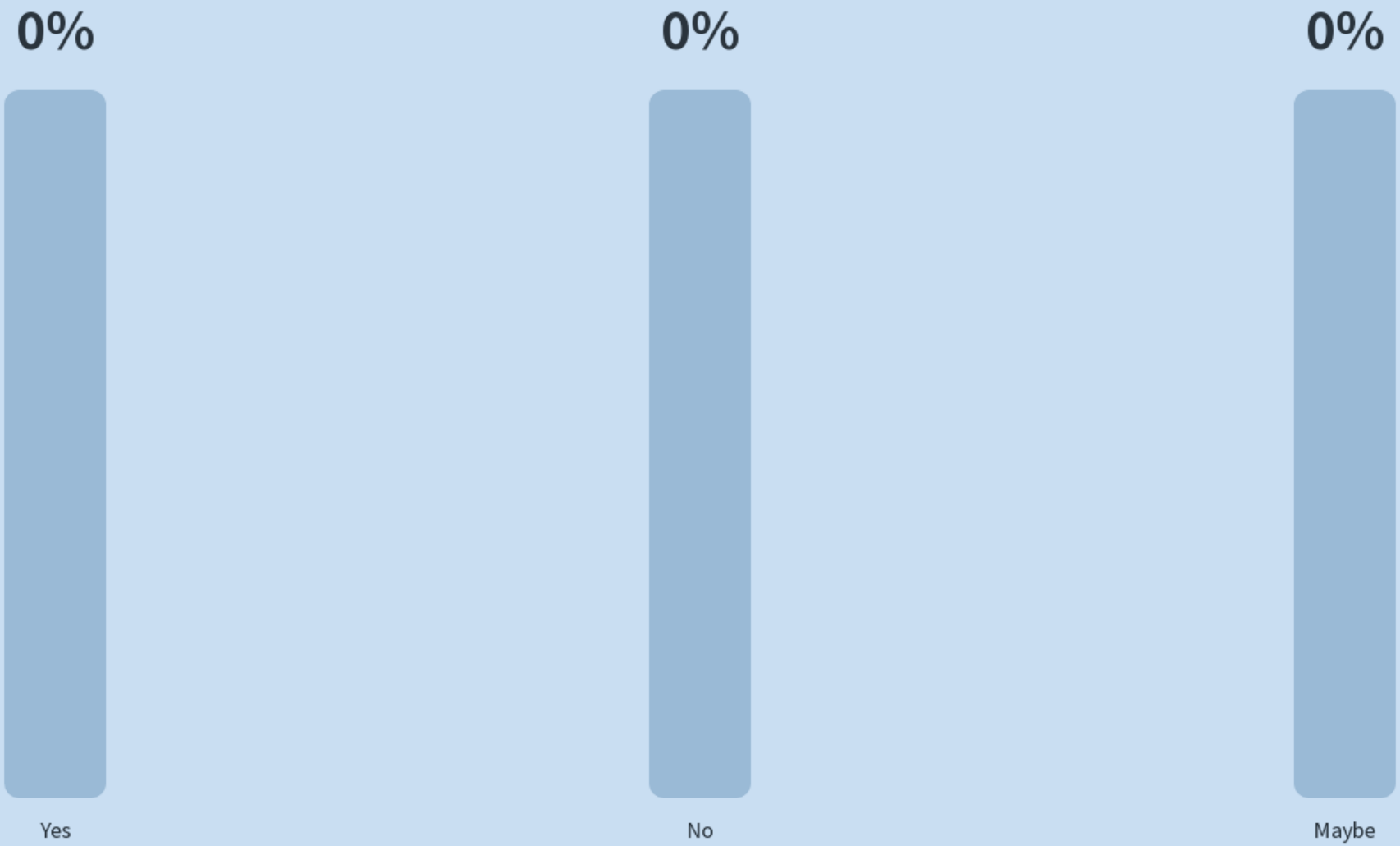
- Requirements gathering
- Systems / Architectural Design
- Implementation / Coding / Integration
- Testing
- Evolution:
  - Delivery and Release - Deployment
  - Maintenance



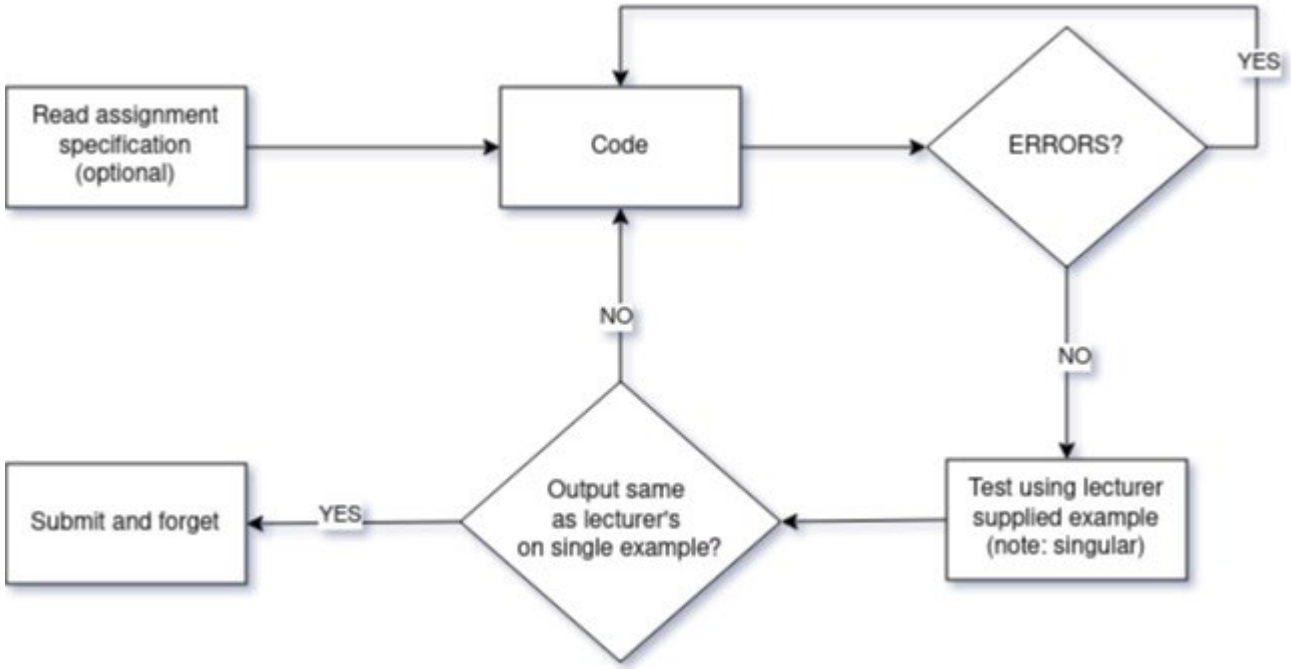
Student Programming Assignment Process Model



Does this process (Student Assignment) Work?



Student Programming Assignment Process Model

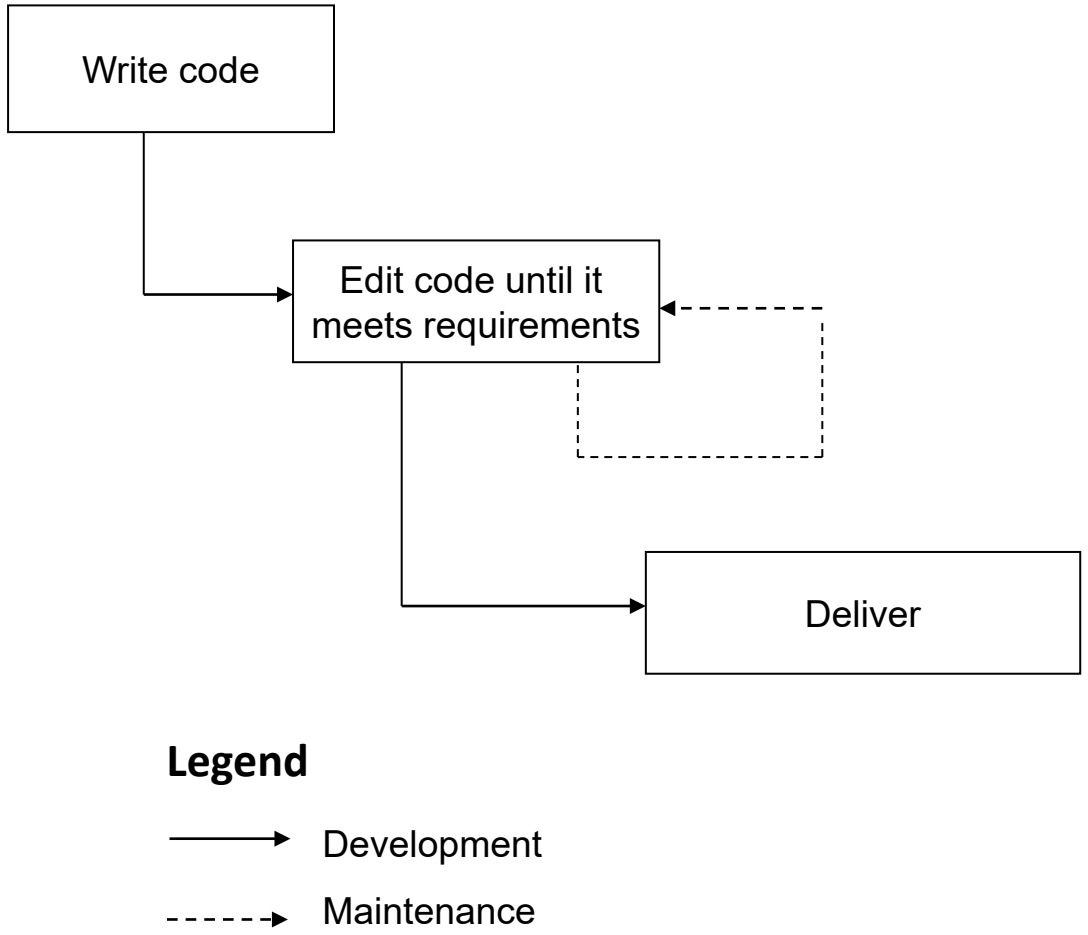


Problems?

- Submitted code tested using different examples
- Student submits something different to what lecturer wants
- Student gets marked down for “poor design”

Code and Fix

- Also known as ad-hoc software development
- Nice and simple!
- definitely simple, anyway…
- When programming was a new discipline, this is what most people did
- Students use Code and Fix approach.



Code and Fix works up to a point

- In the 1940s and 1950s…
  - systems were small (by today's standards)
  - developers tended to be brilliant
    - only the very highly motivated went into the field
    - often people with higher degrees in mathematics or engineering
  - developers, clients, and end users were often the same person
    - fewer issues around requirements
  - programs often were only ever run once
    - little need for maintenance, configuration management, version control

BUT…

Unplanned, ad-hoc fixes can lead to a system that does not work, or is lacking in security, robustness, or usability – especially if time is short.

1960's Software Crisis

- By the 1960s, computers were getting much more powerful
  - so were the programs
  - customers' and users' expectations were becoming more sophisticated
- But a large majority of software projects:
  - shipped late (or not at all)
  - went catastrophically over budget
  - didn't work
  - or all of the above!

What is Software Crisis? [Readings Online in CANVAS]

The Software Crisis [Readings Online in CANVAS]

Please ask your questions about the SWEN90016 Lecture

Nobody has responded yet.

Hang tight! Responses are coming in.

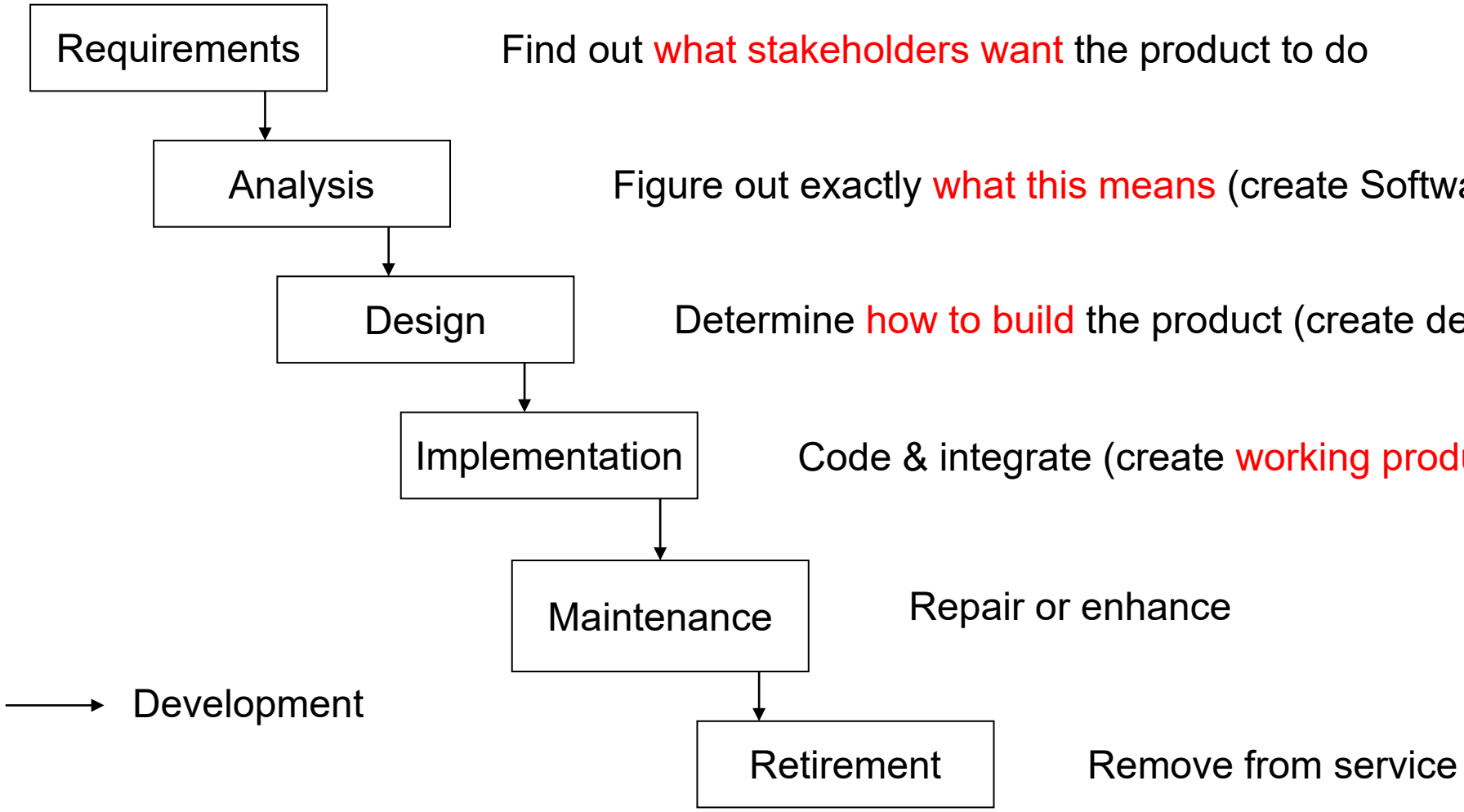
1968-69: NATO conferences

“We build systems like the Wright brothers built airplanes — build the whole thing, push it off the cliff, let it crash, and start over again.”

- The NATO conferences on Software Engineering had an impact still being felt today
  - brought the phrase software engineering into common use
  - leaders in the computing profession resolved to address the problems we’ve mentioned
  - attendees included famous computer scientists such as Edsger Dijkstra, Tony Hoare, Stanley Gill, Alan Perlis, and Peter Naur

1968 NATO Software Engineering Conference [Readings Online in CANVAS]

Waterfall – A plan driven approach (1970)



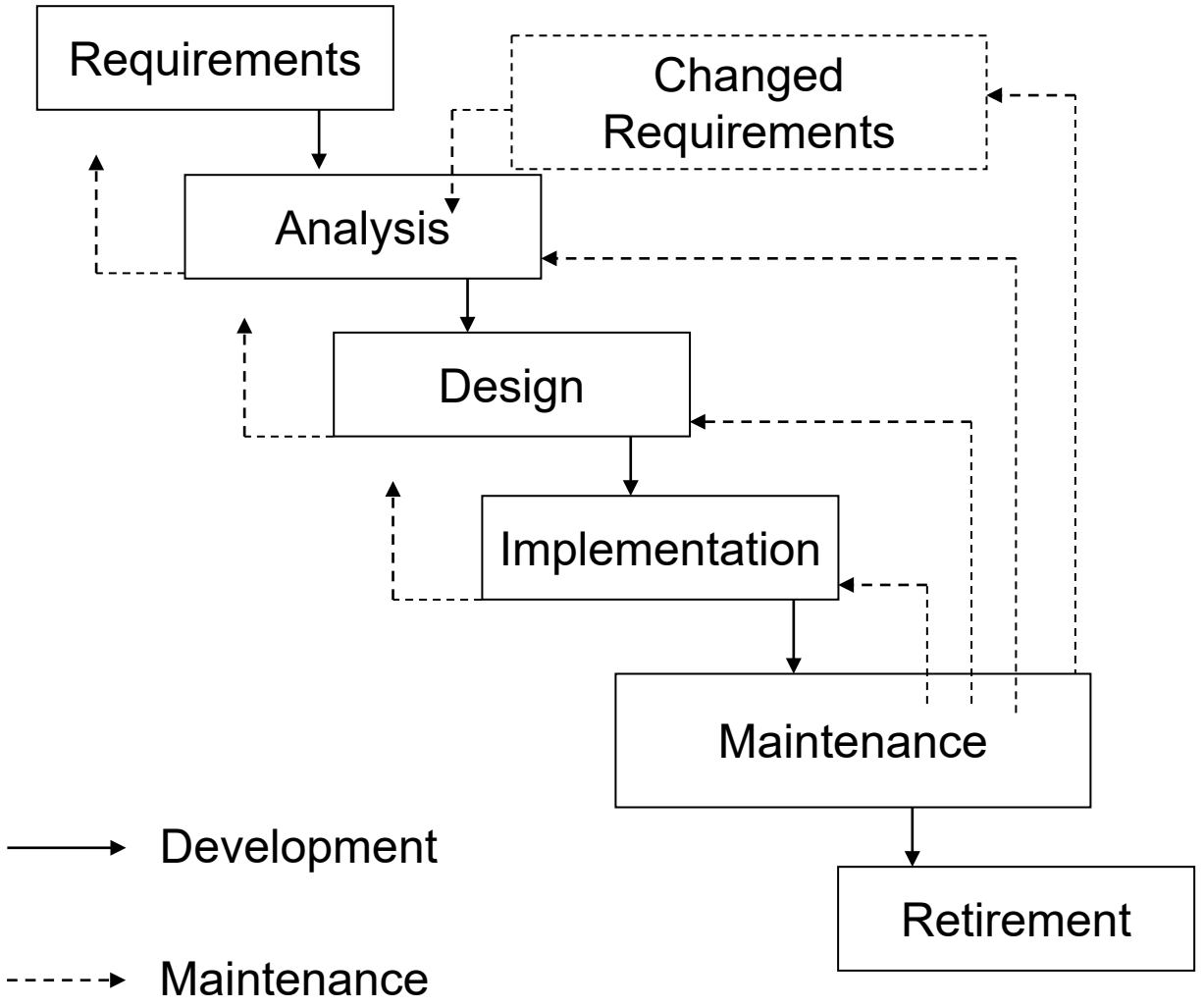
Managing the Development of Large Software Systems: concepts and techniques [Readings Online in CANVAS]

Waterfall – A plan driven approach (1970)

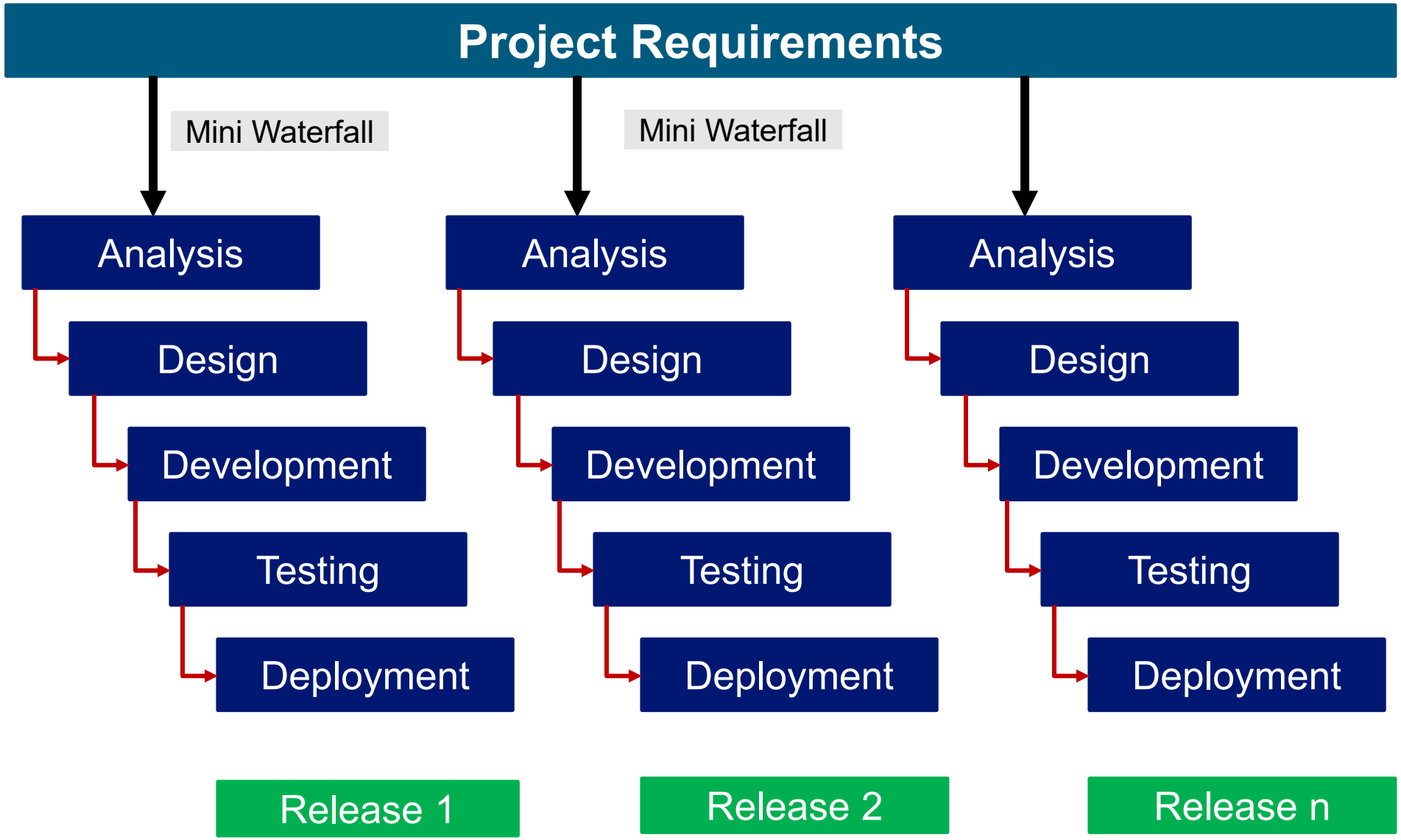
- Development is broken up into distinct phases
- These phases do not overlap, e.g. implementation cannot begin until design is complete
- Each phase has a document of some sort as its output
  - If we consider source code as a special kind of document.
- Not shown: testing and quality assurance activities
  - In general, all documents created would go through QA before development can proceed
  - And of course, QA might find errors.

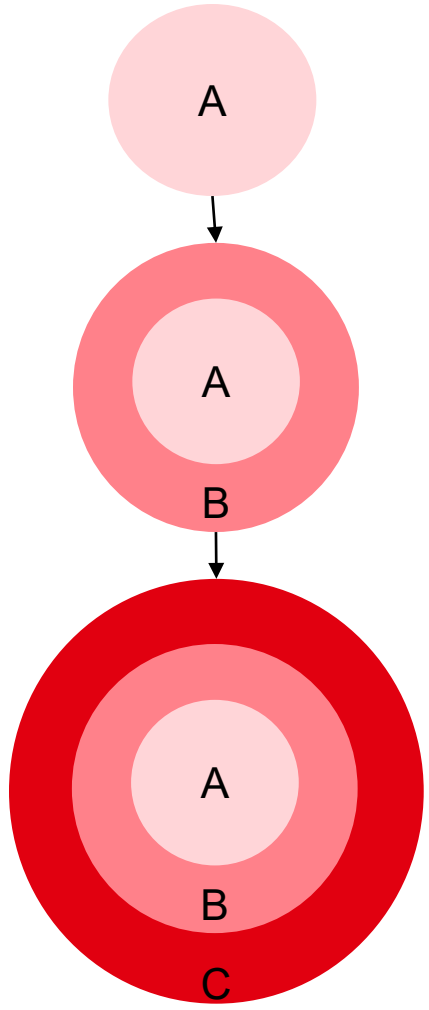
A more Realistic Waterfall – with feedback

- This is a document driven approach
  - Transitions between phases are managed by the production of documentation
    - Notably the SRS and the design documentation
    - If an error is found and the system needs to change, documentation needs to be kept up to date.
    - This very costly
- So, waterfall doesn't work well for projects where:
  - Errors are possible in requirement elicitation, analysis and design
  - Requirements are hard to figure out or subject to change.



Incremental SDLC Model





Incremental SDLC Model

- **Combines elements of Waterfall SDLC in an iterative manner**
- **Each "Mini Waterfall" phase produces deliverable increments of the software**
  - Basic features added in first increment
  - Supplementary features added in subsequent increments
- **SDLC model useful to iteratively construct partial implementations of a system**
- **Each subsequent release of a mini waterfall will incrementally add functions until all designed functionalities are implemented**

https://ieeexplore.ieee.org/document/1204375

Incremental SDLC Model

Advantages:

- Develop major requirements first
- Risk of software development spread across multiple increments
- Lessons learnt from each increment can complement future increments
- Each release of the mini waterfall phase delivers an operational increment
- Initial product delivery is faster
- Reduces the risk of failure

Disadvantages:

- Requires good planning and design
- Designing the increments may not be straightforward
- Requires early definition of requirements to identify the increments
- Model is rigid and does not allow for iterations within each increment or “mini waterfall”

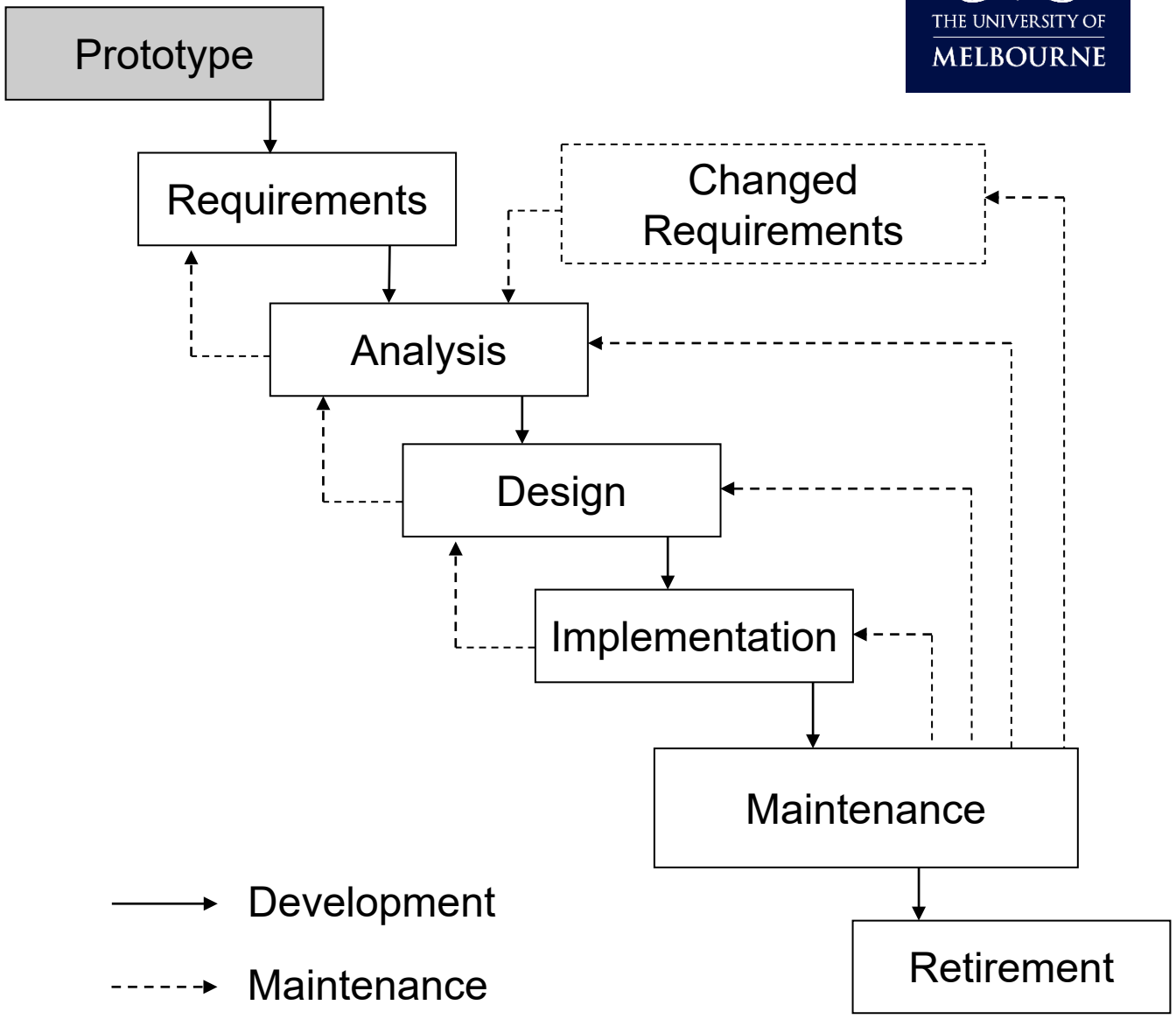
Problems with documentation

- Very few real-world projects are insulated from change
- Even if customers really do know what they want
- Business environment, regulatory environments, etc. are beyond the control of developers and customers alike
- Hard to quarantine projects from changes in the technical environment: operating systems, libraries, network speed etc.

Prototyping

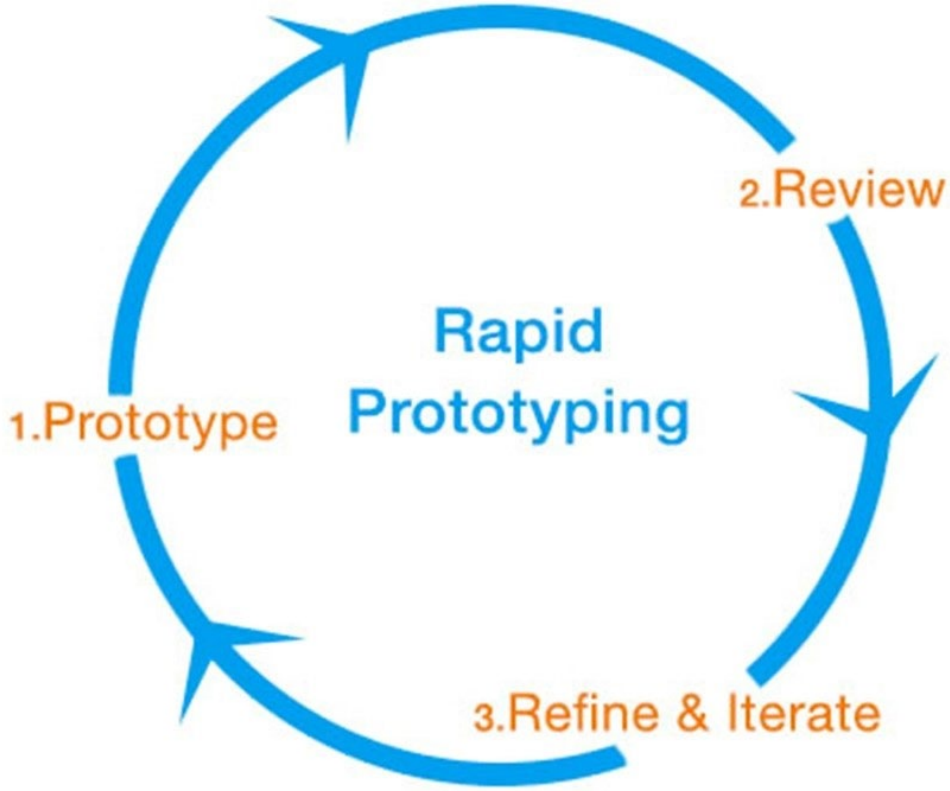
- Key idea: build a prototype of software to help nail down requirements
- Proposed by Royce in 1970
- But:
  - can be expensive
  - single prototype might not be enough
- Need for multiple successive prototypes inspired Spiral

Managing the Development of Large Software Systems [Readings Online in CANVAS]

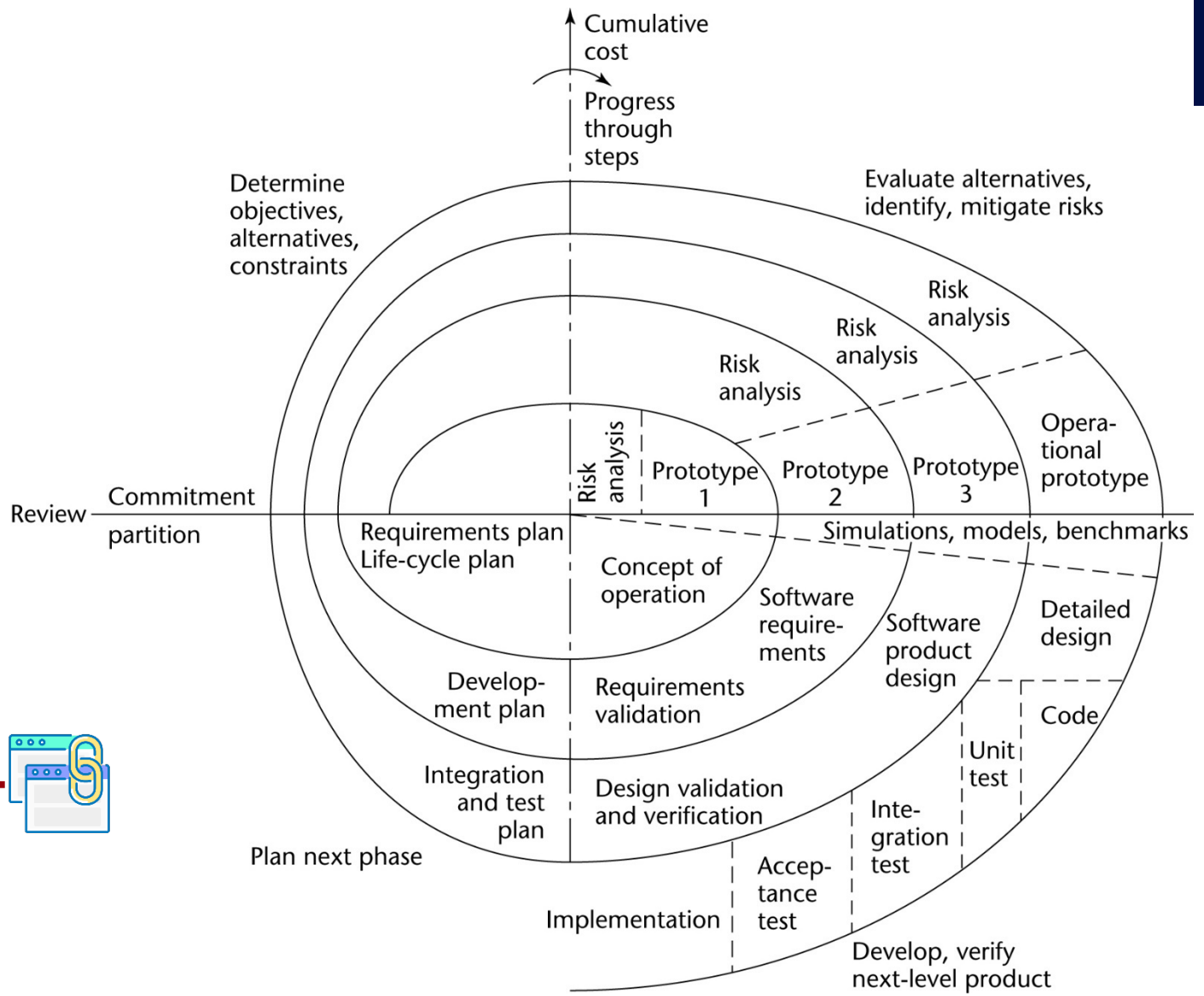


Rapid Prototyping

- Evolved from Prototyping
- Rapid prototyping = working model that is functionally equivalent to a subset of the product
- It is client-oriented, where the client get to provide feedback on the prototype before moving on

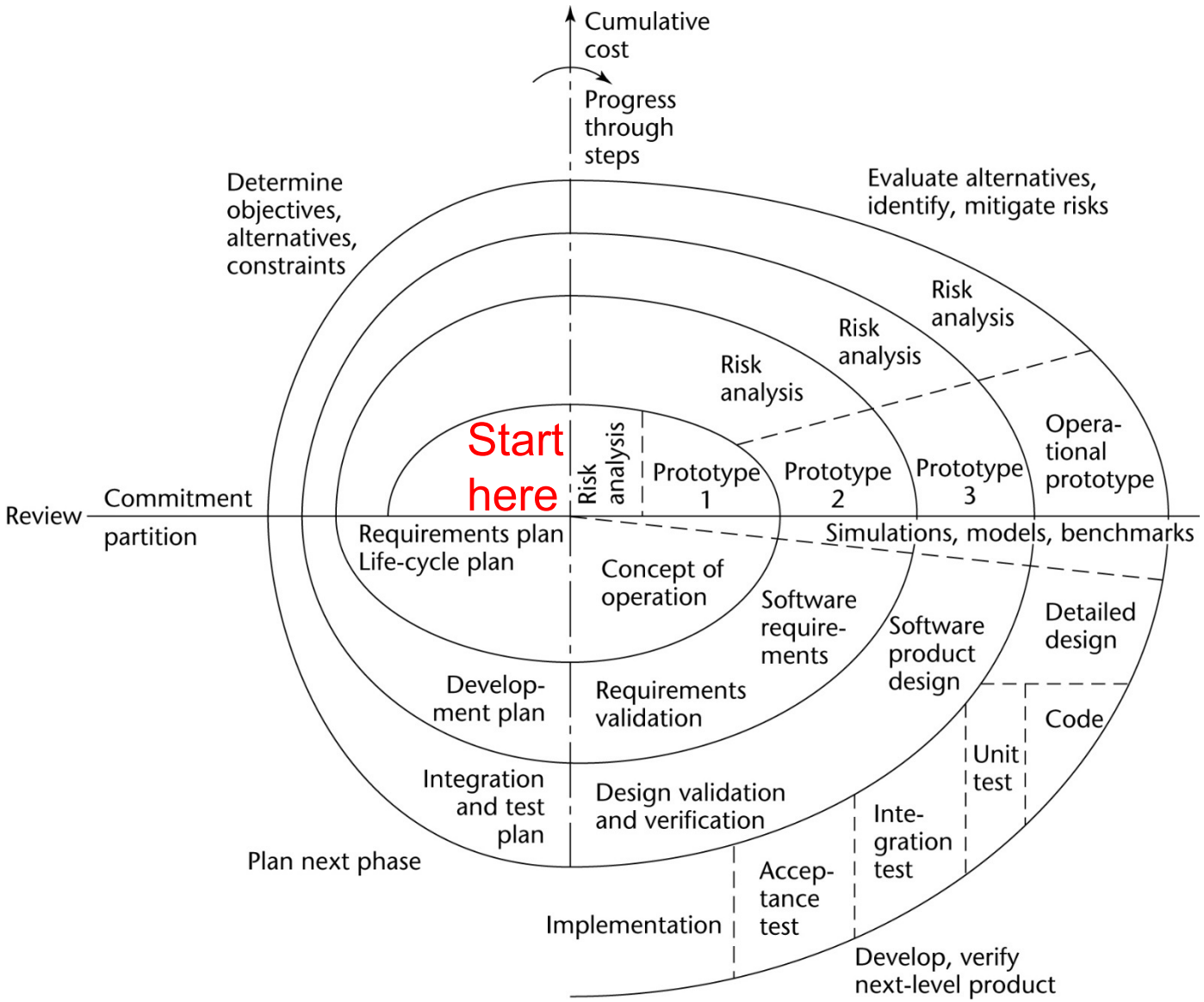


Spiral Model (1986)

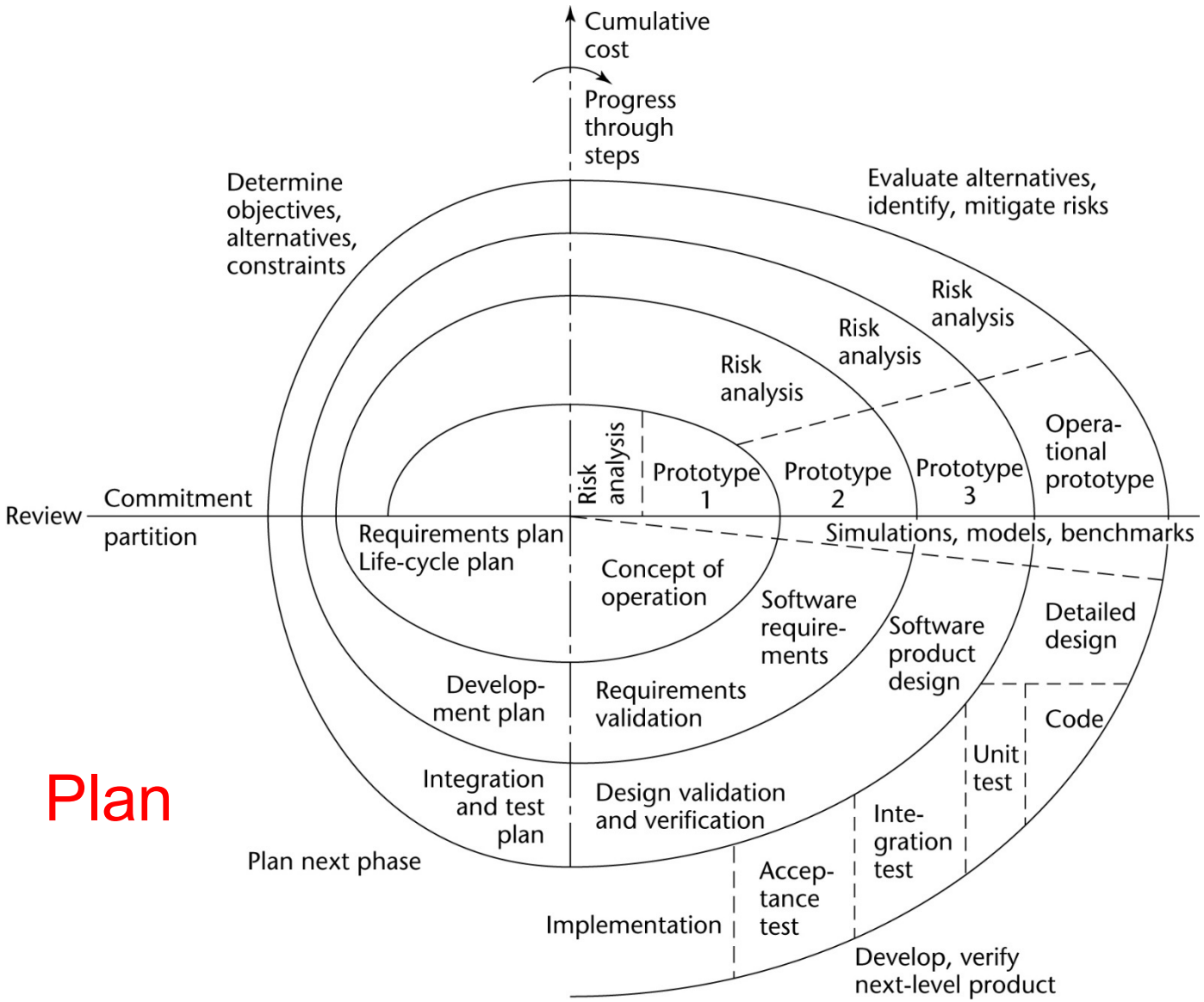


The Spiral Model of Software Development and Enhancement [Readings Online in CANVAS]

Spiral Model (1986)

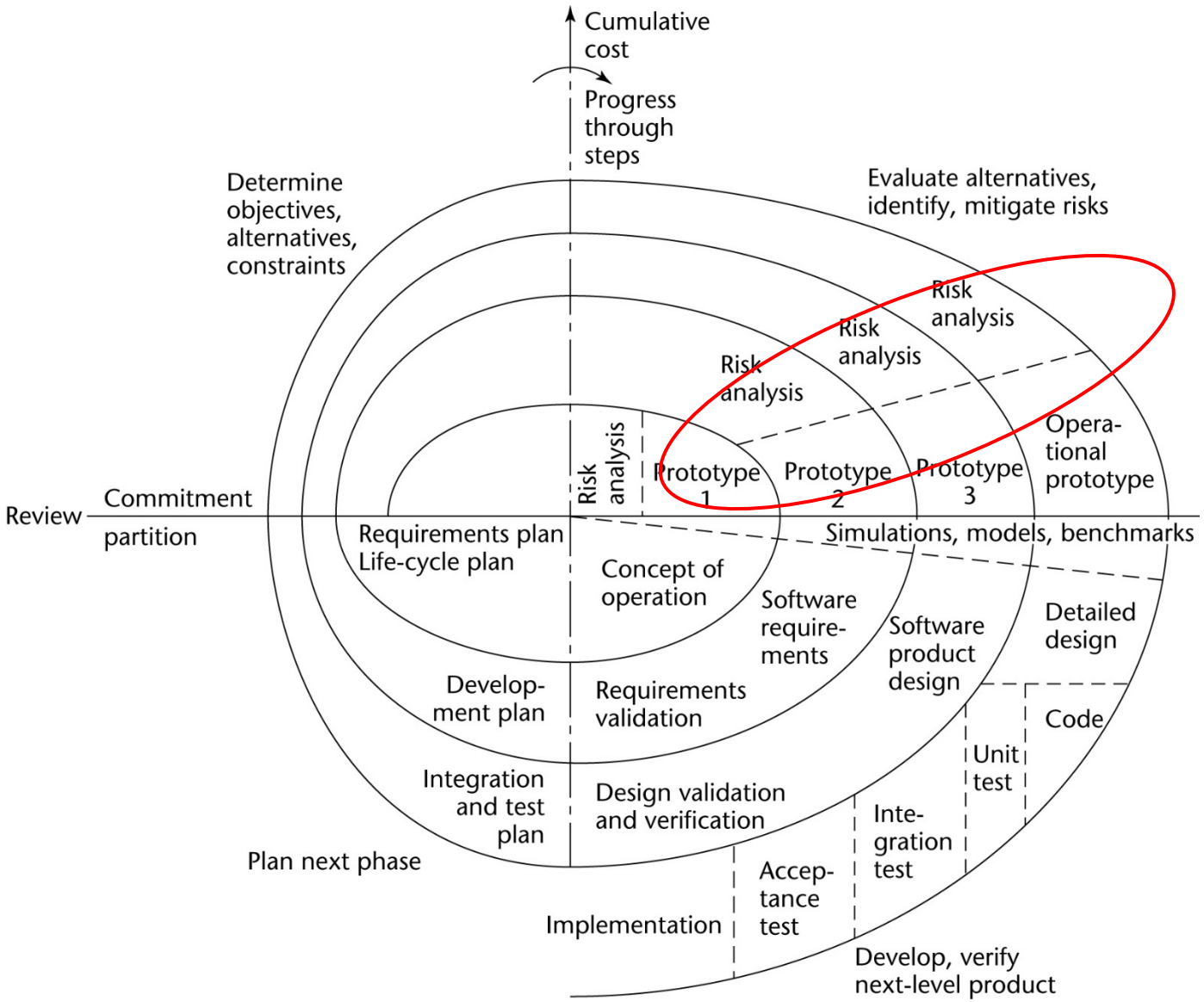


Spiral Model (1986)



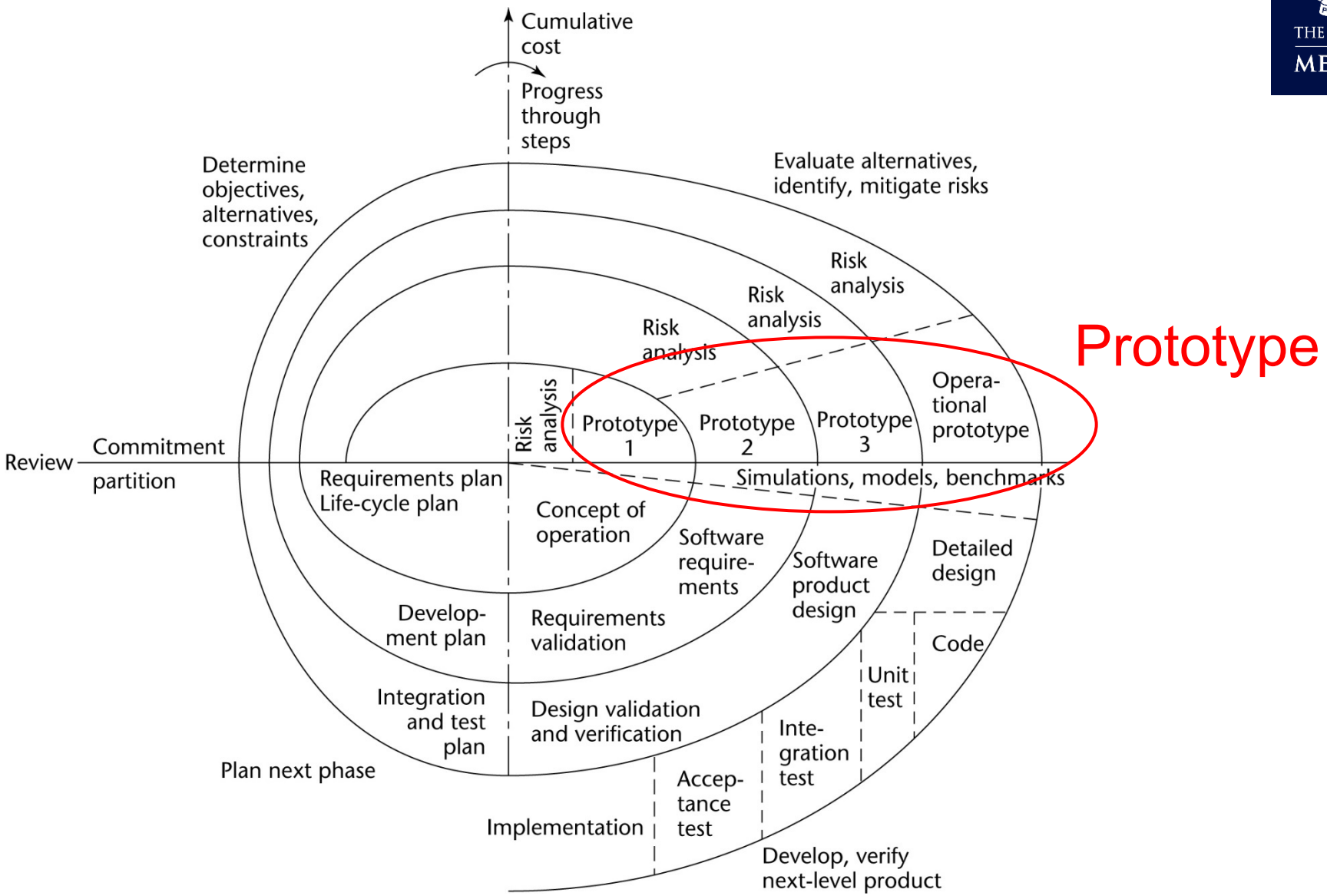
Plan

Spiral Model (1986)



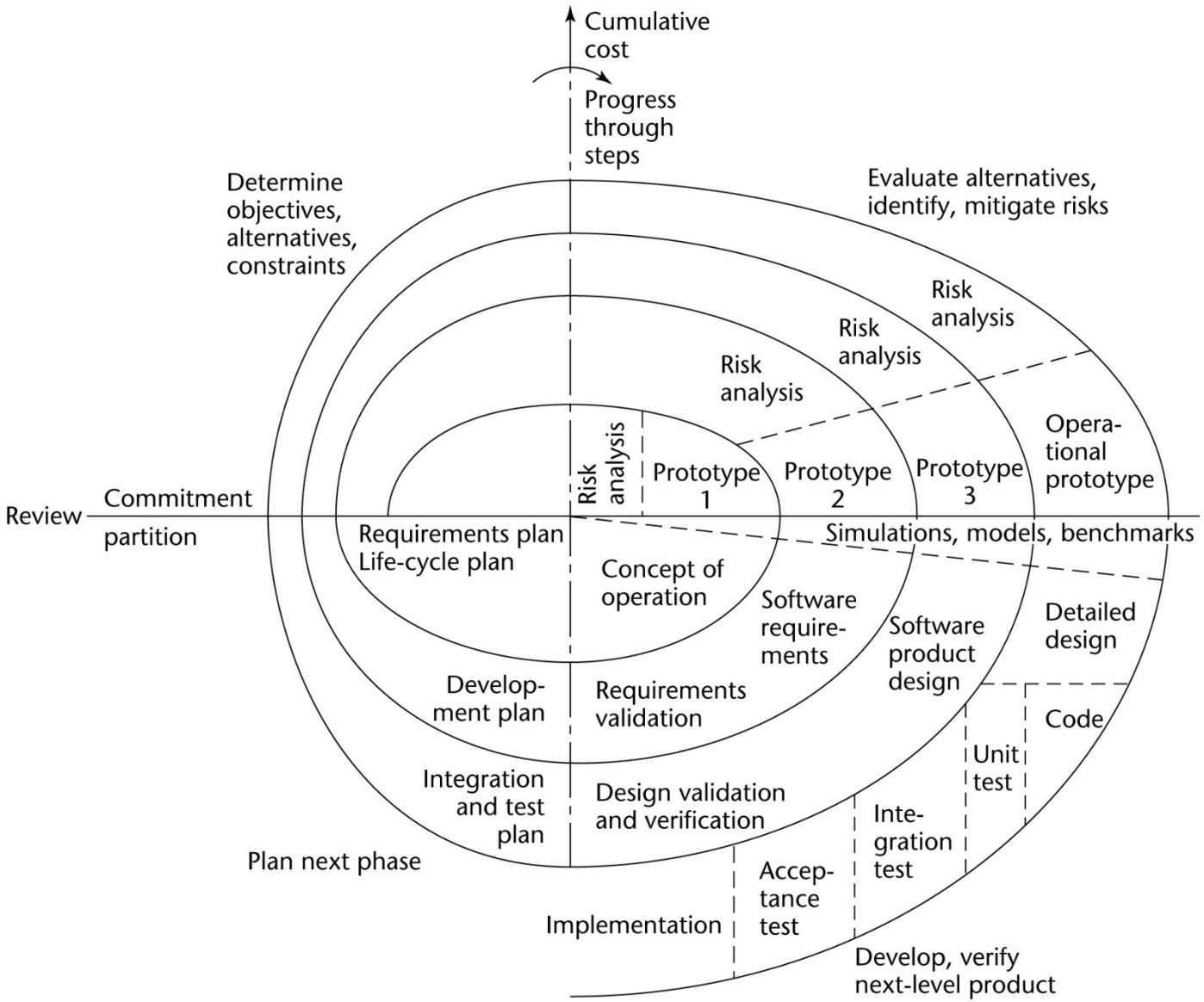
Risk Analysis

Spiral Model (1986)



Prototype

Spiral Model (1986)



Build and validate

Please ask your questions about the SWEN90016 Lecture

Nobody has responded yet.

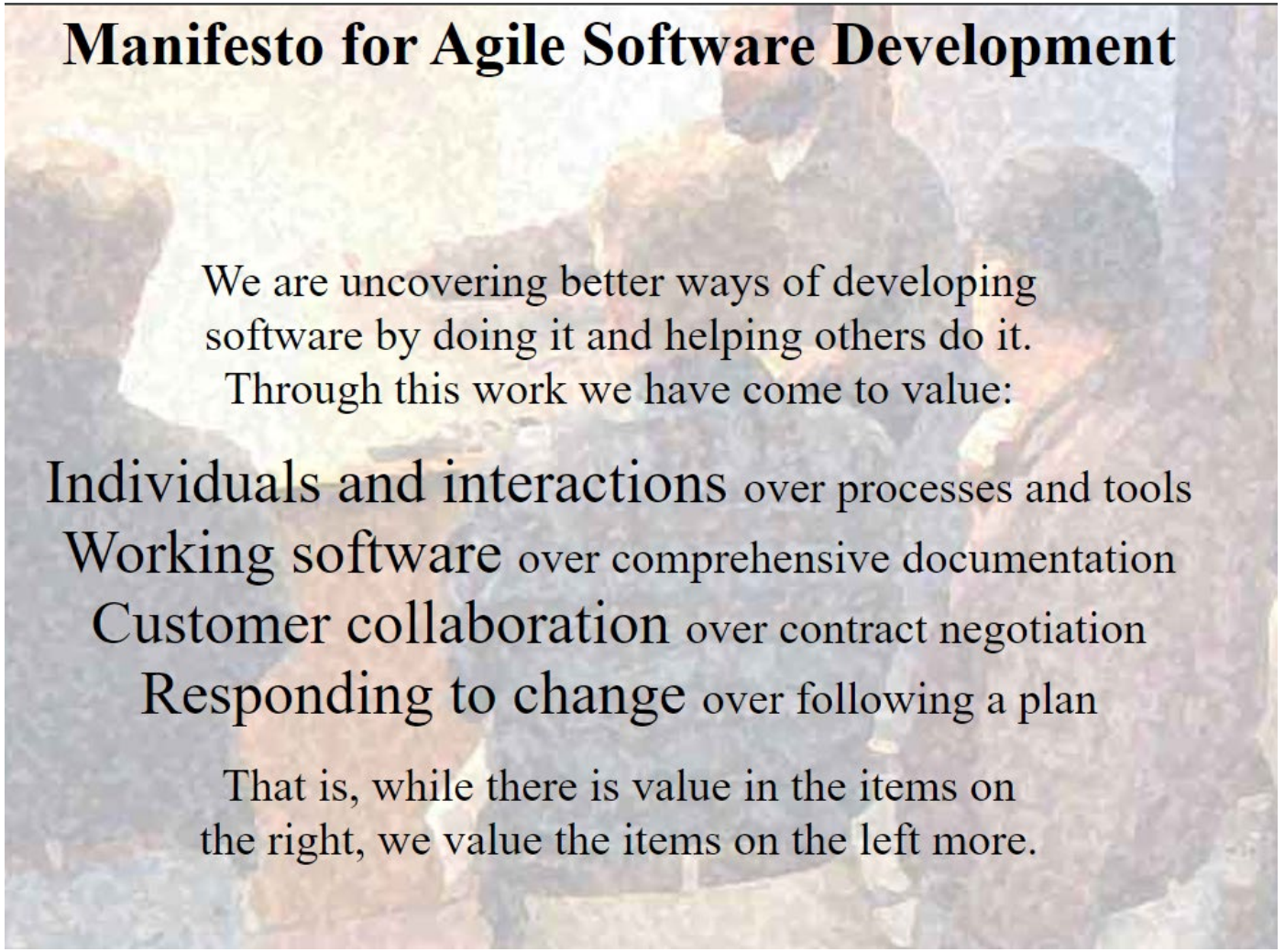
Hang tight! Responses are coming in.

Agile family of processes (2001)

- A group of seventeen like-minded software engineers met at the Snowbird resort in Utah
- Shared interest in so-called **lightweight** process models that deemphasized document-driven software construction
- Not a single process model, but rather a set of core values
- These values are expressed in the Agile Manifesto

History of the Agile Mainfesto [Readings Online in CANVAS]

Agile Manifesto



History of the Agile Mainfesto [Readings Online in CANVAS]

Agile Principles

1. Our highest priority is to satisfy the customer through early and continuous delivery of valuable software.
2. Welcome changing requirements, even late in development. Agile processes harness change for the customer's competitive advantage.
3. Deliver working software frequently, from a couple of weeks to a couple of months, shorter timeframes is the preference.
4. Business people and developers must work together daily throughout the project.
5. Build projects around motivated individuals. Give them the environment and support they need and trust them.
6. The most efficient and effective method of conveying information to and within a development team is face-to-face
7. Working software is the primary measure of progress.
8. Agile processes promote sustainable development. The sponsors, developers, and users should be able to maintain a constant pace indefinitely.
9. Continuous attention to technical excellence and good design enhances agility.
10. Simplicity - the art of maximizing the amount of work not done - is essential.
11. The best architectures, requirements, and designs emerge from self-organising teams.
12. At regular intervals, the team reflects on how to become more effective, then tunes and adjusts its behaviour accordingly.

Principles behind the Agile Manifesto [Readings Online in CANVAS]

Agile Mindset – An example of Agile Product Dev

-

https://www.youtube.com/watch?v=2NFH3VC6LNs

Agile Mindset – what have we learnt?

- What have we seen?
  - Understanding
  - Collaboration
  - Flexibility
  - Feedback Loops
  - Most importantly – adapt to change….
- What haven't we seen?
  - No upfront planning
  - No detailed documentation
  - No step-by-step predefined process
  - No handoff between people.

Cultivating an Agile Mindset [Readings Online in CANVAS]

Agile Process Assumptions

For example:

- The project is sufficiently small to be assignable to a single, small-size, single-location development team.
- The user can be made available at the development site or can interact promptly and effectively.
- The project is sufficiently simple and non-critical to disregard or give little consideration to non-functional aspects, environmental assumptions, risks and other such factors.
- Requirements verification before coding is less important than early release.
- New or changing requirements are not likely to require major code refactoring and rewrite, and the people in charge of product maintenance are likely to be the product developers.
- *Do we want the critical parts of safety or mission critical systems built through agile processes?*

However:

- Agile is not a binary notion: agility can be adjusted through modified processes esp. re requirements

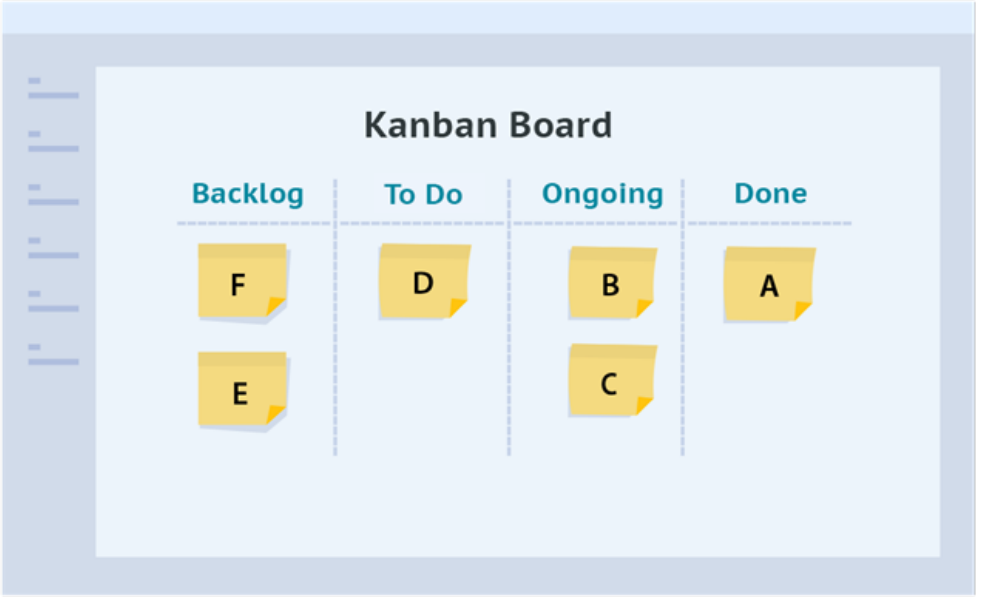
- Page 54, Requirements Engineering by Axel van Lamsweerde, Wiley 2009.

Agile Frameworks

Kanban (カンバン and 看板)

- It means Billboard
- started as a lean manufacturing method by Toyota.
- visualising your work
- limiting work in progress, and
- maximising efficiency.

Kanban teams focus on reducing the time taken to take a project from start to finish.

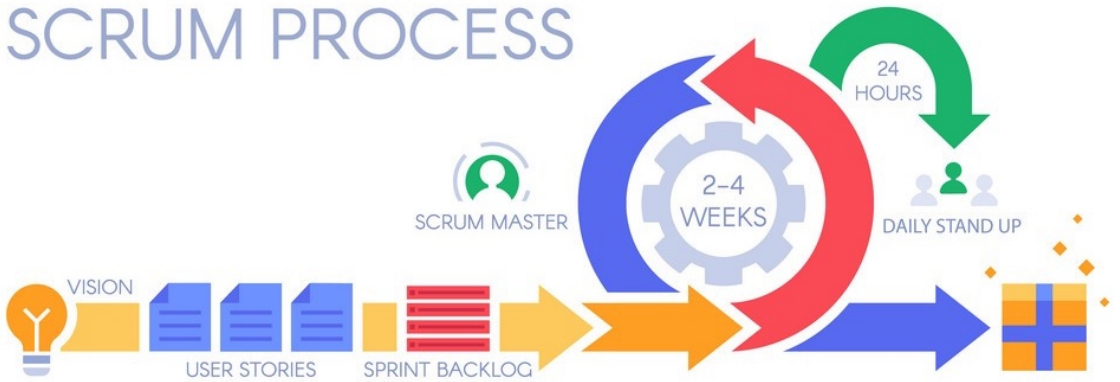


Kanban Guide - an Introduction to Kanban Method [Readings Online in CANVAS]

Agile Frameworks

Scrum

- The term originated in 1986
- Jeff Sutherland and Ken Schwaber presented their paper on Scrum Development Process in a Conference in 1995.
- helps teams work together to develop, deliver and sustain complex products.
- Everything is divided into timeframes

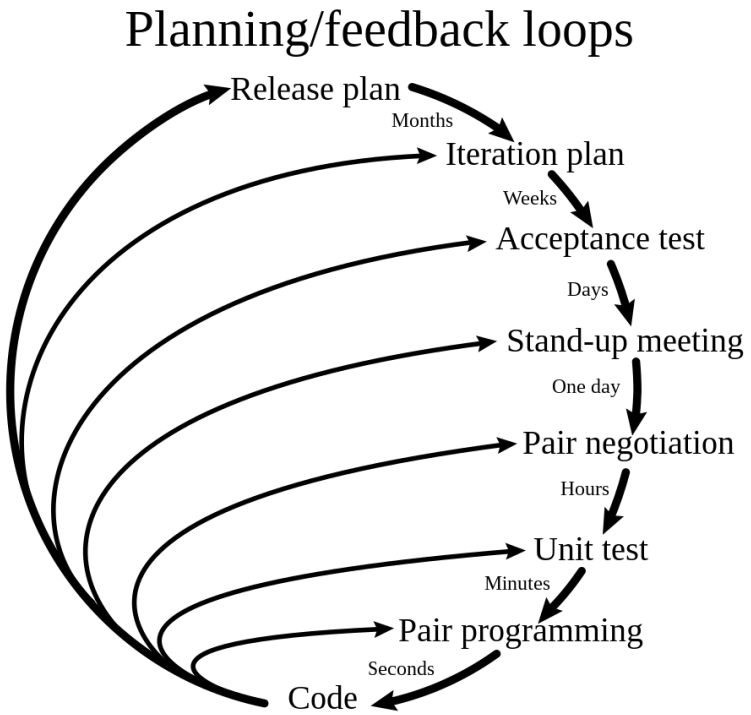


A Short History of Scrum [Readings Online in CANVAS]

Agile Frameworks

eXtreme Programming (XP)

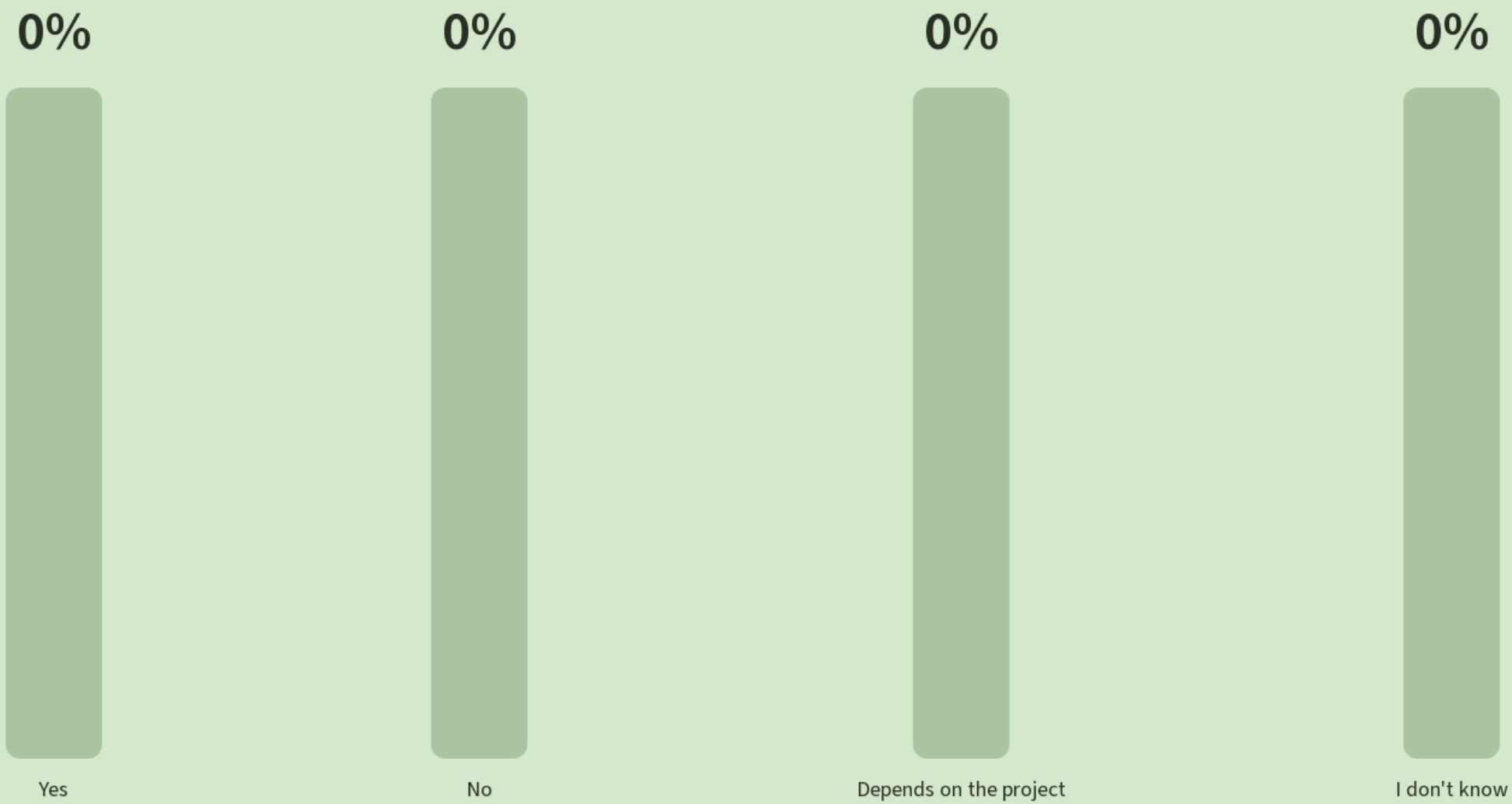
- Kent Beck developed the framework in late 90's
- frequent releases in short development cycles
- intended to improve productivity and
  - introduce checkpoints at which new customer requirements can be adopted.
- Pair Programming



eXtreme Programming (XP) [Readings Online in CANVAS]

Agile Glossary - What is Extreme Programming? [Readings Online in CANVAS]

Can we use Agile and Scrum Interchangeably ?



Scrum Framework

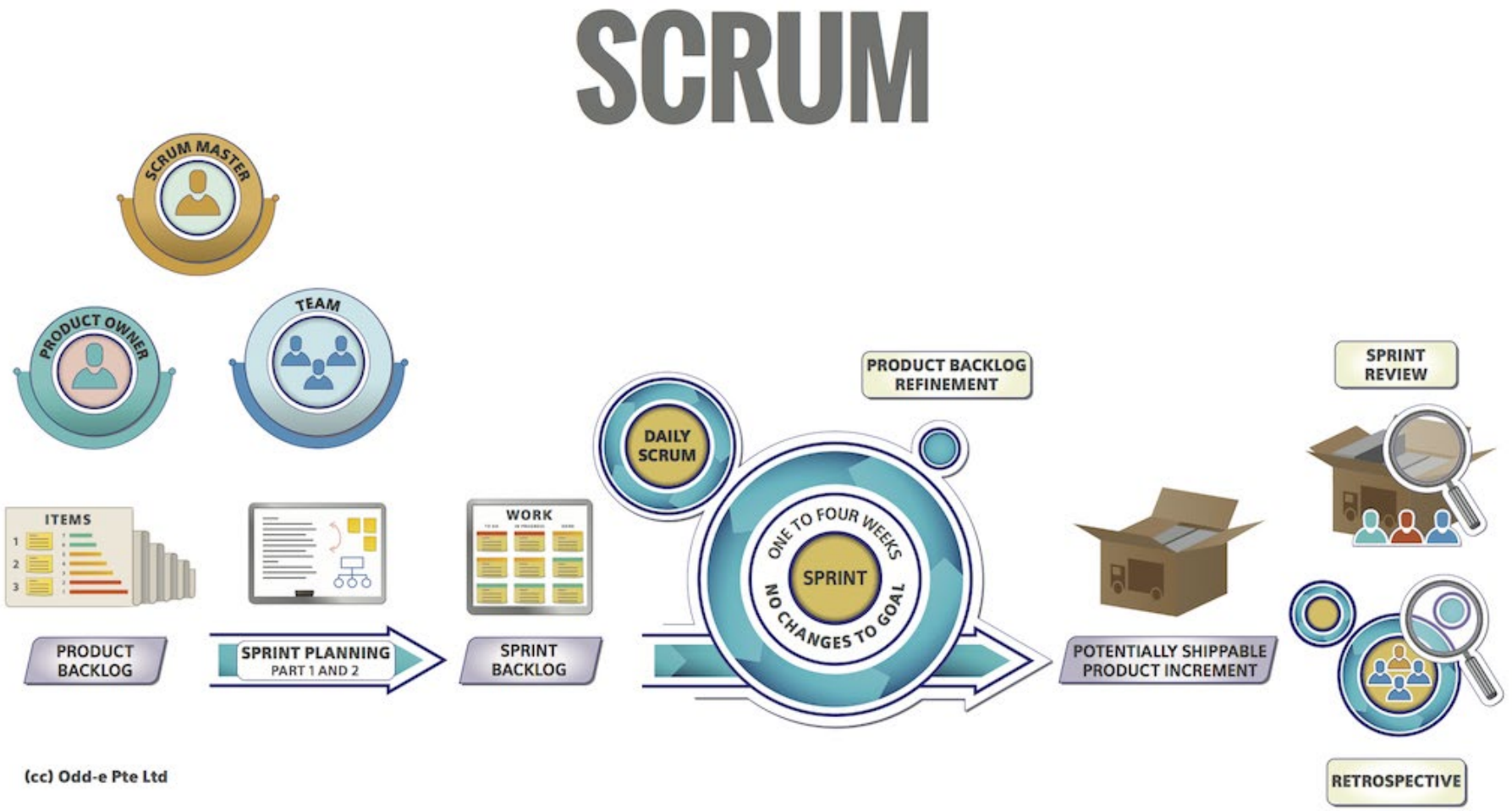
Scrum guide defines Scrum as “A lightweight framework that helps people, teams and organizations generate value through adaptive solutions for complex problems.”

Scrum Premier defines Scrum as “Scrum is an iterative, incremental framework for projects and product or application development.”

So:

It's not a programming method.
It's not even specific to IT.
It's a way to organise groups of people to deliver something.

Scrum in one Picture



Picture Taken from Scrum Overview

Scrum Foundations

- Scrum is founded on empiricism and lean thinking.
- Empiricism asserts that knowledge comes from experience and making decisions based on what is observed.
- Lean thinking reduces waste and focuses on the essentials.
- Iterative, incremental approach to optimise predictability and to control risk.
- Scrum engages groups of people who collectively have all the skills and expertise to do the work and share or acquire such skills as needed.

The 2020 Scrum Guide [Readings Online in CANVAS]

Scrum Pillars

Scrum is standing on three major pillars

Transparency

- Emergent and visible to those who work
- Transparency enables inspection.
- Inspection without transparency is misleading and wasteful.
- Scrum Artefacts help with transparency

Inspection

- Goals must be inspected frequently and diligently to detect potentially undesirable problems.
- Scrum Events help with inspections

Adaptation

- Process should be adjusted if deviated
- Minimise the deviation
- Self management

The 2020 Scrum Guide [Readings Online in CANVAS]

Scrum Documentation

Different types of documentation generated within Agile Model

- Product Backlog
- Definition of Done
- Sprint Backlog
- User Stories
- Acceptance Criteria
- Burndown Charts
- Release Plans
- Retrospective Documents
- Technical Handover Document
- User Manuals

Scrum Accountabilities

Scrum Master

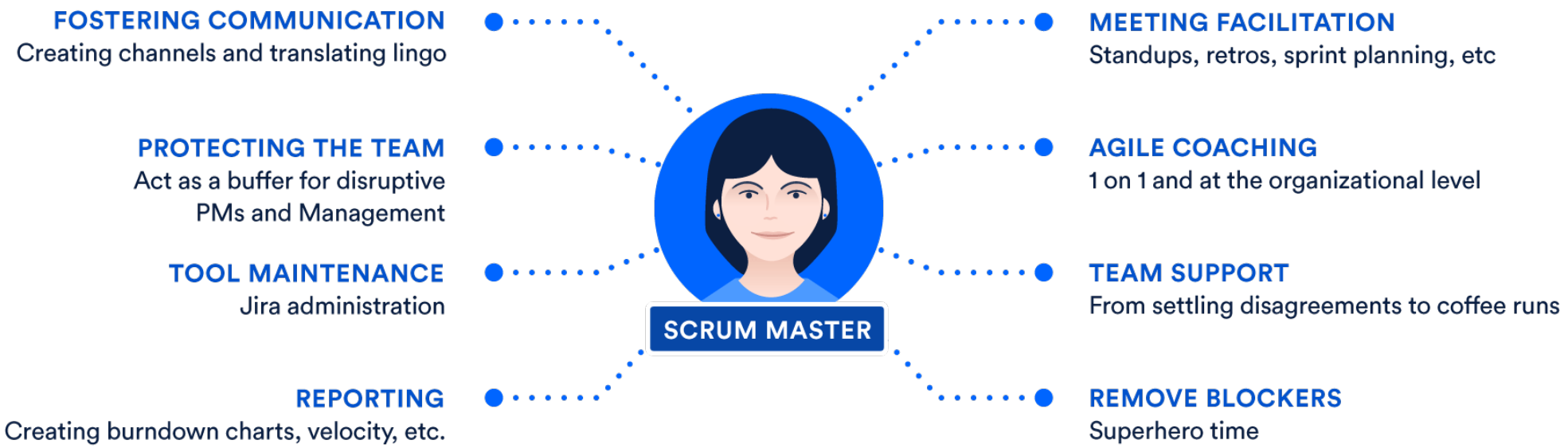


Image Courtesy: https://www.atlassian.com/agile/scrum/scrum-master

Product Owner

Image Courtesy: https://ecoemil.medium.com/tech-stuff-scrum-scrum-roles-the-product-owner-daf8476cc473

Developers/ Development team

Image Courtesy: https://www.atlassian.com/agile/scrum/roles

A tech startup wants to develop a mobile app which helps users track their fitness routines. The startup has a general idea of the app's features but expects the requirements to evolve based on user feedback. What SDLC model do you propose and why?

Nobody has responded yet.

Hang tight! Responses are coming in.

The owner of a bookstore wants you to develop software for the shop. The owner knows what the software will do exactly. What SDLC do you propose and why?

Nobody has responded yet.

Hang tight! Responses are coming in.

A government agency needs a new software system to manage public records. The requirements are not fully defined, and stakeholders want to see a visual representation of the system before finalizing them. What SDLC do you propose and why?

Nobody has responded yet.

Hang tight! Responses are coming in.

Summary

- There are many SDLCs around with organisations typically favouring a blend of Formal and Agile approaches.
  - Formal approaches are useful when
    - Projects where the customer has a very clear view of what they want
    - Projects that will require little or no change to requirements
    - Software requirements are clearly defined and documented
    - Software development technologies and tools are well-known
    - Large scale applications and systems developments

Please ask your questions about the SWEN90016 Lecture

Nobody has responded yet.

Hang tight! Responses are coming in.

Case Study

- PayPal acquired by eBay, Inc in 2002
- Initially small, nimble, rapidly built payments solutions
- Positioned as a leader in the payments industry
- By 2008, innovation began to slow, product release cycles lengthened

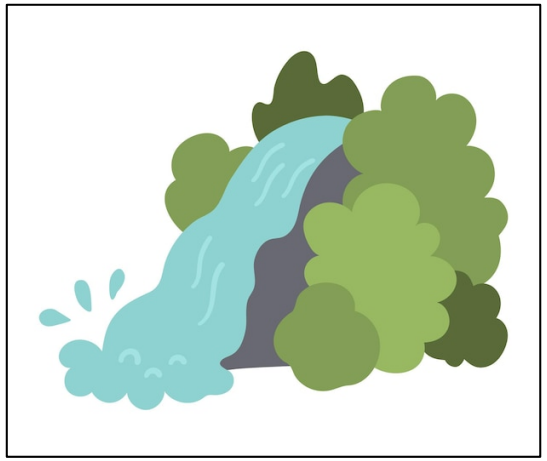
PayPal Enterprise Transformation Whitepaper [Readings Online in CANVAS]

Key Drivers of Slowdown 1/2

- **Detailed Requirements Documents**
  - Required for annual planning
  - Took months to create
  - Often outdated before coding began
- **Complex Quarterly Planning**
  - Technology Quarterly Planning (TQP) developed
  - Added flexibility but also complexity
- **Domain Bottlenecks**
  - 85 unique domains created
  - Unintended creation of bottlenecks

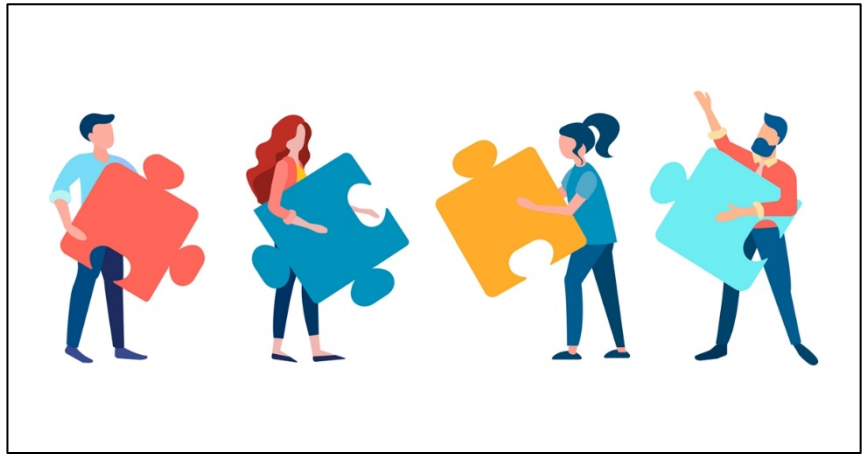
Key Drivers of Slowdown 2/2

- **Developer Context Switching**
  - Reduced productivity due to allocation to multiple projects
- **Traditional Waterfall Development**
  - Forced agile practices into waterfall cycles
- **Integration Testing Cycles**
  - Rigorous testing required
  - 6-week integration and regression test cycles



Organizational Frustration

- Developers: Delayed results, direction conflicts
- Product Managers: More documentation than development
- Executives: Slow process, low confidence
- Customers: Experience not keeping pace with industry
- Critical need for change to retain customers and talent



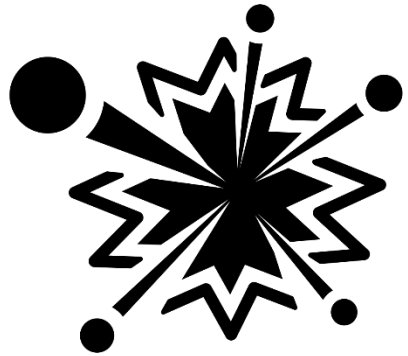
Organizational Frustration

- Developers: Delayed results, direction conflicts
- Product Managers: More documentation than development
- Executives: Slow process, low confidence
- Customers: Experience not keeping pace with industry
- Critical need for change to retain customers and talent

- Radical Transformation initiated

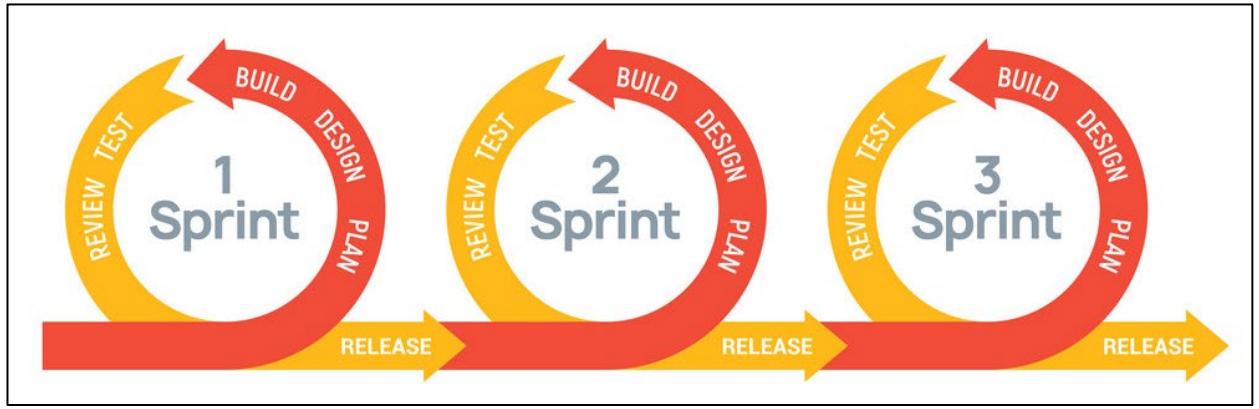
“Big Bang” Approach

- Chosen against industry wisdom
- Need for rapid, interdependent changes
- Formation of over 300 cross-functional teams
- Adoption of Agile Scrum, Customer Driven Innovation (CDI)
- Use of Scaled Agile Framework, Large Scale Scrum
- New structure: 17 Product Lines, 31 Sub-Product Lines, 87 Delivery Groups



Enterprise Sprint Cadence

- Pre-transformation: high variability in practices
- “Big Bang” alignment: same two-week sprint cycle
- Short cycles to focus on user stories
- Sprints started and ended mid-week for global collaboration
- Sprint numbers used for cross-team communication



Lifecycle Management Tool

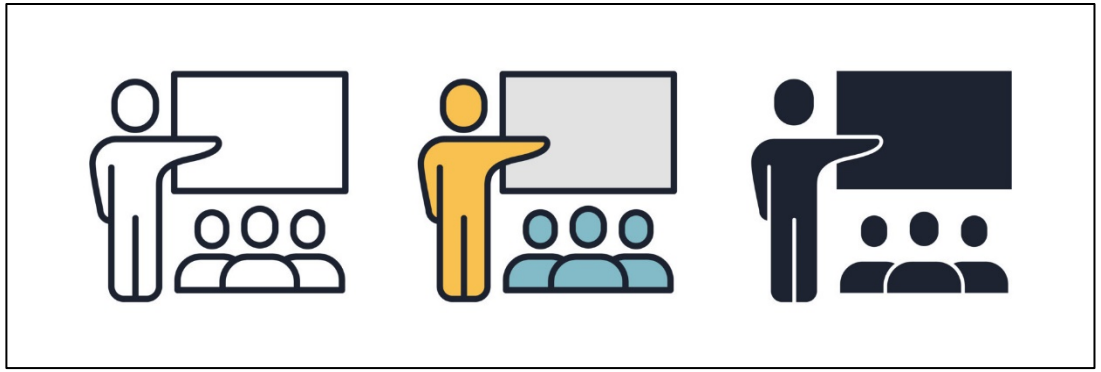
- Pre-transformation: variety of tracking approaches
- Evaluation of available agile tools
- Single tool chosen for all teams, with enterprise partnership
- Alignment with sprint cadence, new product structure
- Training and coaching support provided

What was… will be…

What was…	Will be…
Project Driven	Product Aligned
Waterfall	PayPal Agile
Feature oriented requirements	Customer driven solutions
Project based planning	Planning by team, group, and product
People assigned to Projects	Stable Teams aligned to Product
15 Dedicated Teams	Delivery Teams and Delivery Groups
Phases and Phase Exits	Roadmaps, Release Plans and Sprints

Rollout and Challenges

- Employee adaptation to large-scale change
- Communication of new teams, structures, and standards
- Varied challenges: lack of Agile experience, existing practices
- Support through videos, training, and feedback channels



Results of Transformation

- Drastic change in leadership focus and operational practices
- Shift to “Team Sprints” and product roadmaps
- Agile Scrum practices for planning and delivering work
- Faster response to changing product priorities
- Customer insights incorporated into dynamic solution concepts

References

Refer to Readings Online in CANVAS

Lecturer Identification

Dr. Rajesh Chittor Sundaram

The University of Melbourne (Australian University)
PRV12150/CRICOS 00116K